TUT NCV PROGRAMMING TRAINING REPORT

Day 1 – Monday, 19 October 2026: Getting Started

The first day focused on the NC(V) Level 2 course, problem-solving skills, version control and an introduction to Python. We looked at the course topics and how the week's work links to what learners need to know. We also discussed how to break a problem into smaller steps before writing a program.

We learned how Git and GitHub help us keep track of changes to our work. The sessions covered creating a repository, making a copy of the course files, and using commands such as git add, git commit and git push. This showed why saving a file on a computer is not the same as sending it to GitHub.

We then used Scratch ideas to link block-based coding to Python. In Thonny, we looked at the editor and shell and wrote a simple welcome program using print(). We also covered common coding errors and the need to test work. The day gave me a clear base for planning, writing and saving Python tasks.

Day 2 – Tuesday, 20 October 2026: From Problem to Program

The second day focused on planning a program before writing code. We worked with Polya's problem-solving steps and the Program Development Life Cycle (PDLC). These steps showed how to understand a task, plan a solution, write the code and check that it works.

We used Input, Processing and Output (IPO) charts to show what data a program needs, what it must do with that data and what result it should display. We also worked with flowcharts and algorithms. Tracing steps helped us see how a program moves from one instruction to the next and where errors may occur.

The day linked these plans to basic Python work, including variables and simple code. We also looked at user needs and test cases. I found the planning tools useful because they give learners a clear path to follow instead of starting with code at once. They can also help learners explain their answers in practical tests.

Day 3 – Wednesday, 21 October 2026: Data, Input and Calculations

The third day focused on writing Python programs that take in data and perform calculations. We covered the main data types: int, float, str and bool. We also worked with variables, naming rules and math signs such as +, -, *, /, //, % and **. We learned why brackets matter when Python works out an answer.

In Thonny, we used input() to get values from the user and int() or float() to change text into numbers. Our tasks included finding an area and working out an average. We also used string methods, f-strings and two-decimal output to make results easier to read.

Later, we worked with the math module, tested a hypotenuse program and recorded test results. We also designed a Python practical task and its marking memo. This was useful for my teaching because it linked the code to clear instructions, fair marks and tests that show whether a learner's program works.

Day 4 – Thursday, 22 October 2026: Decisions and Loops

The fourth day focused on making decisions and repeating steps in Python. We started with Boolean values and signs used to compare values, such as ==, !=, > and <=. We then used if, elif and else to make a program choose the right result based on a condition.

One main task was a mark classifier. It checked a mark and placed it in the right band. We also looked at how to check that a mark is within the valid range. This showed why the order of conditions matters and why programs should be tested with values at the edge of each range.

After that, we worked with while and for loops. We used counters, totals, stop values and range() to repeat tasks without writing the same code many times. The practice tasks included a valid-mark average and a program that worked with cubes and totals. The day helped me see how to teach decisions and loops in small steps, with tests after each change.

Day 5 – Friday, 23 October 2026: Functions, Lists and Debugging

The final day focused on functions, lists and finding errors in Python code. We learned how to use def to create a function, pass values through parameters and return a result. We also looked at local variables and why a value made inside a function may not be used outside it.

We then used lists to store several marks in one place. We worked with list items and ways to find totals, averages and the highest mark. These skills were brought together in a learner-mark program with a menu. The task showed how a large program can be split into smaller parts that are easier to read and test.

The last part covered syntax, runtime and logic errors. We used Thonny to trace code, find faults, correct them and test again. The week ended with a focus on the final project, assessment work and Portfolio of Evidence. The training gave me more practice with Python and useful ways to guide beginners through coding tasks.